

3. Découverte de Wireshark

Quelles sont les interfaces auxquelles vous avez accès ?

A quoi correspond chacune d'elles ?

Sous Windows : Interfaces typiques

Quand tu ouvres Wireshark, tu peux voir des noms comme ceux que tu m'as cités :

1. Ethernet / Connexion au réseau local

- Carte réseau câblée (câble RJ45 branché).
- Si ton PC est branché par câble → c'est cette interface qui transporte ton trafic.

2. Wi-Fi

- Carte réseau sans fil (Wireless).
- Si tu es connecté par Wi-Fi → tout ton trafic Internet passe par cette interface.

3. VMware Network Adapter / VirtualBox Host-Only Network

- Interfaces **virtuelles** créées par des logiciels de virtualisation.
- Elles servent pour les communications entre ton PC hôte et tes machines virtuelles.
- Exemple : si tu fais tourner une VM Linux dans VMware, tu verras son trafic réseau sur cette interface.

4. Npcap Loopback Adapter

- Interface spéciale installée par **Npcap** (le moteur de capture).
- Sert à capturer le trafic **local** entre applications de ton PC (exemple : ping 127.0.0.1, ou accéder à http://localhost).

5. Connexion SSH/UDP/Remote capture

- Interfaces "spéciales" que Wireshark affiche quand :
 - tu utilises un **tunnel SSH** pour capturer à distance,
 - tu fais du **remote capture** via Wireshark,
 - ou tu analyses directement du trafic encapsulé (UDP, TCP).
- Ce ne sont pas des cartes réseau physiques, mais des **points de capture** spécifiques.

Sous Linux : Interfaces typiques

Sous Linux, les noms sont différents, mais l'idée est la même :

1. eth0, eth1, ... (ou enpXsY, eno1 selon la version de Linux)

- Interfaces Ethernet (câblées).
- Utilisées quand ton PC est branché en RJ45.

2. wlan0, wlan1, ...

- Interfaces Wi-Fi.
- Utilisées quand tu es connecté en sans-fil.

3. lo (loopback)

- Interface "loopback".
- Trafic interne à ton PC (127.0.0.1, ::1).

- Exemple : si tu fais tourner un serveur web local (http://localhost:8000), ce trafic circule dans lo.

4. **tun0, tap0, ...**

- Interfaces VPN (OpenVPN, WireGuard, etc.).
- Le trafic chiffré de ton VPN passe dedans.

5. **docker0, vethXXXX, br0** (si Docker/VM installés)

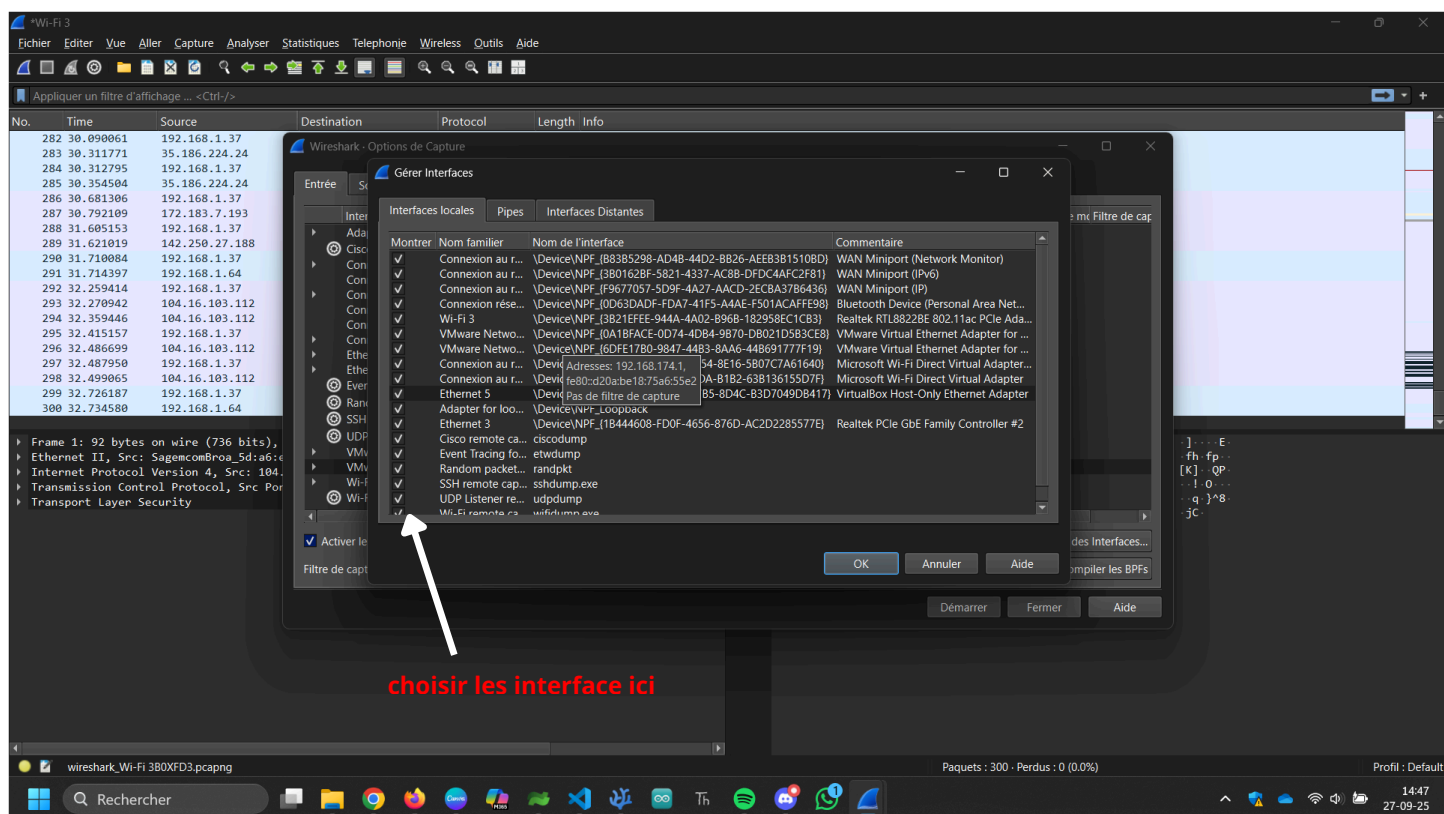
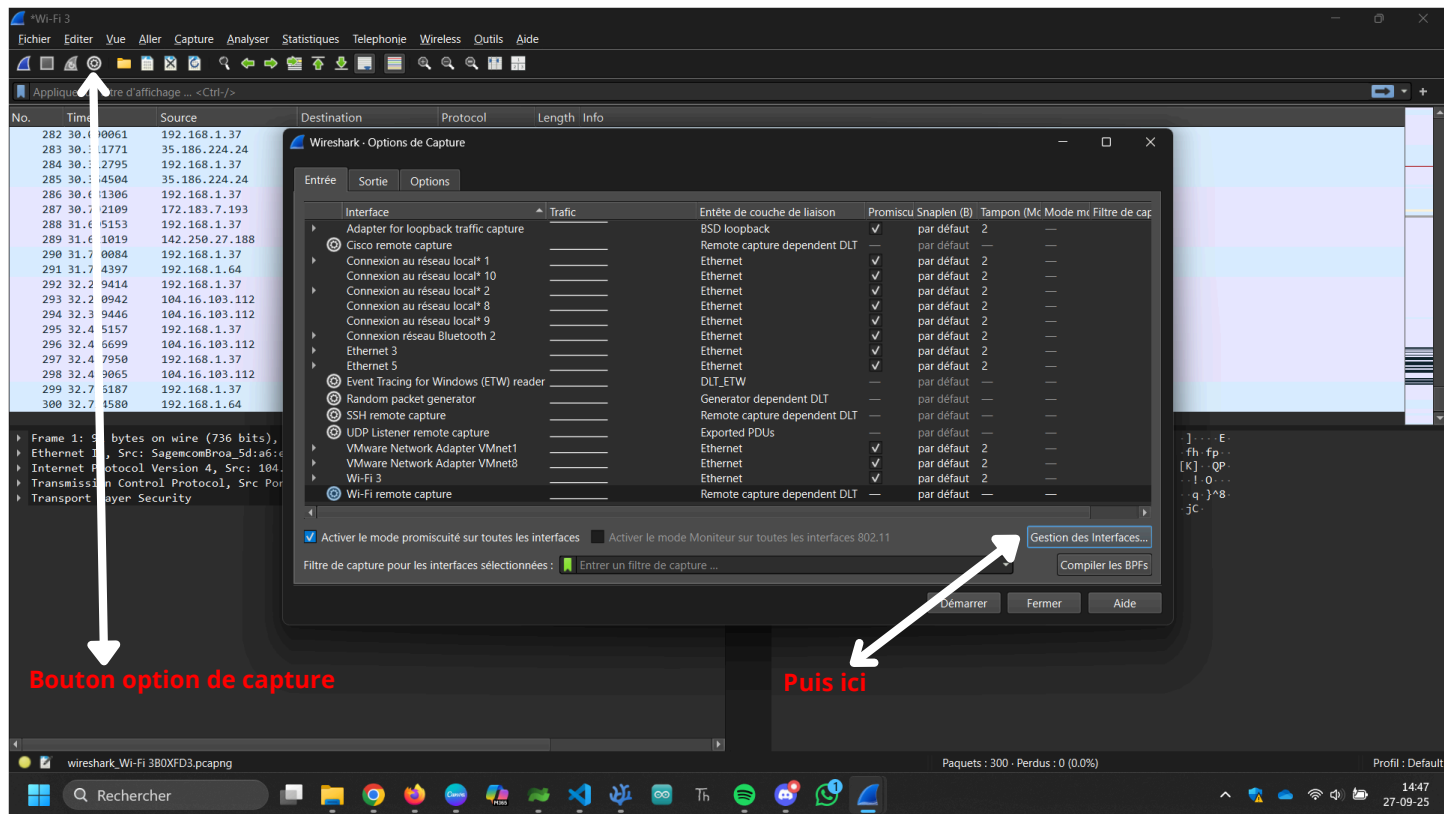
- Interfaces virtuelles créées pour les conteneurs (Docker) ou la virtualisation (KVM, QEMU, VirtualBox).
 - Permettent aux VM/containers de communiquer avec ton PC hôte.
-

Comment faire pour capturer le trafic depuis toutes les interfaces à la fois ?

Dans l'interface graphique de Wireshark

1. **Ouvre Wireshark.**
2. Dans l'écran d'accueil, tu as la liste de toutes les interfaces.
3. En haut, tu vois un bouton "**Capture → Options...**" (ou l'icône d'engrenage).
4. Là, tu peux **cocher plusieurs interfaces** à la fois.
 - Si tu coches toutes, Wireshark va capturer simultanément sur chacune.
5. Clique sur **Start** → tu verras les paquets des différentes interfaces défiler dans une seule fenêtre.

💡 Astuce : dans une capture multi-interface, Wireshark ajoute une colonne **Interface** pour que tu voies d'où vient chaque paquet.



En dessous de la partie menu, l'interface se délimite en trois zones rectangulaires, de taille ajustable. Quelle information contient chacune d'elle ?

Sélectionnez différents éléments et voyez en quoi l'affichage est modifié.

The screenshot shows the Wireshark network protocol analyzer interface. The top bar indicates the interface is 'Wi-Fi: en0'. Below the toolbar, there's a filter bar with 'Apply a display filter ... <3%>'. The main area is divided into three zones:

- Zone 1:** A table listing captured packets. The table has columns: No., Source, Destination, Protocol, Src port, Dst port, and Info. The table shows 27 packets. The 15th, 16th, and 17th packets are highlighted in red, indicating they are selected.
- Zone 2:** A detailed view of the selected packet (No. 15). It shows the frame structure: Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Transport Layer Security. The details are expanded for the Transport Layer Security section.
- Zone 3:** A hex dump of the selected packet's raw bytes. It shows the hexadecimal representation of the data, along with its ASCII interpretation.

At the bottom of the interface, it shows 'wireshark_Wi-Fi_20200917173844_16ey7x.pcapng', 'Packets: 27 · Displayed: 27 (100.0%)', and 'Profile: Default'.

Zone 1 : Liste des paquets capturés

- **Emplacement :** tout en haut, juste sous la barre d'outils.
- **Contenu :**
 - Chaque ligne = **un paquet** capturé.
 - Colonnes typiques : **No, Time, Source, Destination, Protocol, Length, Info.**
 - Exemple : paquet TCP entre ton PC et un serveur web.
- **Interaction :**
 - Si tu **cliques sur un paquet**, il est **sélectionné** → les autres zones (2 et 3) changent pour afficher ses détails.
 - Tu peux **trier les colonnes**, **appliquer un filtre**, etc.

Zone 2 : Détail du paquet sélectionné

- **Emplacement** : zone du milieu.
- **Contenu** :
 - Affiche le paquet **décomposé par couches** : Ethernet → IP → TCP/UDP → Application.
 - Chaque ligne peut être **dépliée ou repliée** pour voir les champs détaillés (ex. adresse IP source, port TCP, flags).
- **Interaction** :
 - Cliquer sur une ligne de cette zone **met en surbrillance** les octets correspondants dans la zone 3 (octets bruts).
 - Permet de **comprendre exactement chaque champ du paquet**.

Zone 3 : Octets du paquet (raw data)

- **Emplacement** : zone du bas.
 - **Contenu** :
 - Montre le paquet **en hexadécimal et ASCII**.
 - Chaque octet correspond aux données réelles envoyées sur le réseau.
 - **Interaction** :
 - Quand tu sélectionnes un champ dans la zone 2, la zone 3 **met en surbrillance les octets correspondants**.
 - Utile pour analyser des détails précis ou pour déboguer des paquets corrompus.
-

Sélectionnez au hasard une trame pour laquelle l'adressage IP est disponible.
Quelle est l'IP source ?

L'IP destination ?

L'adresse MAC source ?

Destination ?

Étape : sélectionner une trame

Dans la zone 1

- (liste des paquets capturés), clique sur **un paquet qui contient IP**.
- Pour être sûr, regarde la colonne **Protocol** → TCP, UDP ou ICMP sont des paquets IP.
- Une fois sélectionné, la **zone 2** (détail du paquet) va montrer plusieurs couches :
 1. Ethernet II
 2. Internet Protocol Version 4 (IPv4) ou IPv6
 3. TCP / UDP / ICMP (ou autre protocole de transport)

Identifier les adresses

Dans la zone 2 :

◆ Couche Ethernet II

- **Source MAC** → "Source"
- **Destination MAC** → "Destination"

◆ Couche IP (IPv4 ou IPv6)

- **Source IP** → "Source"
- **Destination IP** → "Destination"

The screenshot displays the Wireshark network protocol analyzer interface. The top menu bar includes options like 'Fichier', 'Edit', 'Vue', 'Aller', 'Capture', 'Analyser', 'Statistiques', 'Telephonie', 'Wireless', 'Outils', and 'Aide'. Below the menu is a toolbar with various icons for file operations, capture control, and analysis. The main window is divided into three panes: 'Paquets' (Packets), 'Details', and 'Bytes'. The 'Paquets' pane on the left lists captured packets, with packet 24 selected. The 'Details' pane on the right shows the hierarchical structure of the selected packet, including Ethernet II, Internet Protocol Version 4, and User Datagram Protocol. Red arrows point to specific fields: 'address mac source' points to the Source MAC address (00:50:56:c0:00:08), 'address mac destination' points to the Destination MAC address (ff:ff:ff:ff:ff:ff), 'ip source' points to the Source IP address (192.168.92.1), and 'ip destination' points to the Destination IP address (192.168.92.255). The 'Bytes' pane on the right shows the raw data in hexadecimal and ASCII.

Faites le relevé d'une dizaine d'adresses MAC et d'une dizaine d'adresses IP figurant dans les traces Wireshark.

Dans chaque cas, à qui appartiennent ces adresses ?

Dans le cas d'une trame donnée, est-ce que l'adresse MAC destination correspond à la même machine que l'IP de destination ?

Relever les adresses dans Wireshark

Ouvre ta capture Wireshark.

1. **Filtre les paquets IP** pour simplifier la lecture :
2. ip
3. (tu verras uniquement les paquets IPv4/IPv6).
4. Dans la **zone 1** (liste des paquets), note pour une dizaine de paquets :
 - **IP source** et **IP destination**
 - **MAC source** et **MAC destination** (dans la couche Ethernet II, zone 2).

Comprendre à qui appartiennent ces adresses

● IP source / IP destination

- IP = adresse logique sur le réseau (assignée par le routeur ou statiquement).
- Exemple :
 - 192.168.1.10 → ton PC (réseau local)
 - 142.250.190.78 → serveur Google

● MAC source / MAC destination

- MAC = adresse physique de la carte réseau, unique par interface.
- Dans un réseau local :
 - MAC source = carte réseau de la machine qui envoie le paquet
 - MAC destination = carte réseau de la machine qui reçoit le paquet **sur le même LAN**

IP et MAC destination : correspondance ?

● Pas toujours !

- Dans un **réseau local (LAN)** : MAC destination correspond à la machine finale si elle est sur le même LAN.
- Dans le cas d'un **paquet envoyé vers Internet** :
 - MAC destination = **passerelle / routeur local**
 - IP destination = serveur final sur Internet
- Donc sur Internet, l'IP de destination et la MAC de destination **ne correspondent pas à la même machine**, car ton PC envoie les paquets au routeur qui les transmet ensuite.

5. Filtres

Dans votre trace, affichez tous les paquets ARP

Dans le **filtre en haut**, tape :

arp

puis tous les paquets IP.

Dans le **filtre en haut**, tape :

ip

Avez-vous du trafic IPv6 dans votre trace ?

Dans le **filtre en haut**, tape :

ipv6

Utilisez les opérateurs logiques pour afficher tout le trafic IP qui ne contient pas de segment TCP.

Dans le **filtre en haut**, tape :

ip and not tcp

Affichez les messages DNS véhiculés dans des datagrammes UDP.

DNS est généralement encapsulé dans UDP (port 53).

- Filtre :

udp.port == 53

- Tu peux aussi filtrer uniquement DNS avec le protocole :

dns

6. Mise en pratique

Ouvrez la trace [arp_resolution.pcapng](#).

Où se trouve-t'on dans le modèle OSI ?

ARP (Address Resolution Protocol) est un protocole qui **réside entre la couche 2 (liaison) et la couche 3 (réseau)**.

Plus précisément :

- Il fonctionne **au niveau de la couche liaison de données (Ethernet)** pour transmettre les trames.
- Mais son rôle est de **résoudre les adresses IP (couche 3)** en adresses MAC (couche 2).

Donc, en termes simples : **entre couche 2 et couche 3**, parfois on dit "couche 2.5".

Quelles sont les couches encapsulées dans chaque trame ?

Pour **ARP** sur Ethernet :

- **Couche 2 (Ethernet II)**
 - MAC source
 - MAC destination
 - Type = 0x0806 (indique que le protocole est ARP)
- **Couche 3 (ARP)**
 - Adresse IP source et destination
 - Adresse MAC source et destination (dans le message ARP, parfois MAC destination est inconnue)
 - Opcode : request (demande) ou reply (réponse)

Pour chacune des trames, identifiez les adresses sources et destination.

The image shows a Wireshark capture of two ARP frames. The top pane displays a list of captured packets. The bottom pane shows the detailed view of the first frame, which is an ARP request.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	HonHaiPrecis_6e:8b:24	Broadcast	ARP	42	Who has 192.168.0.1? Tell 192.168.0.114
2	0.004081	DLink_0b:22:ba	HonHaiPrecis_6e:8b:24	ARP	46	192.168.0.1 is at 00:13:46:0b:22:ba

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface unknown, id 0

- Ethernet II, Src: HonHaiPrecis_6e:8b:24 (00:16:ce:6e:8b:24), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
 - Destination: Broadcast (ff:ff:ff:ff:ff:ff)
 - Source: HonHaiPrecis_6e:8b:24 (00:16:ce:6e:8b:24)
 - Type: ARP (0x0806)
 - [Stream index: 0]
- Address Resolution Protocol (request)
 - Hardware type: Ethernet (1)
 - Protocol type: IPv4 (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: request (1)
 - Sender MAC address: HonHaiPrecis_6e:8b:24 (00:16:ce:6e:8b:24)
 - Sender IP address: 192.168.0.114
 - Target MAC address: 00:00:00:00:00:00 (00:00:00:00:00:00)
 - Target IP address: 192.168.0.1

Trame	ip source	ip destination	mac source	mac destination
1	192.168.0.114	192.168.0.1	00:16:ce:6e:8b:24	00:00:00:00:00:00
2	192.168.0.1	192.168.0.114	00:13:46:0b:22:ba	00:16:ce:6e:8b:24

Pour chacune des trames, expliquez son contenu.

La requête ARP (ARP Request)

L'ordinateur Avec l'ip **192.168.0.114** se demande : "Qui a l'adresse **IP 192.168.0.1** ? Donne-moi ton MAC !"

Il envoie cette demande **à tout le réseau** (c'est le broadcast).

Dans cette trame, on voit :

- **Sender IP/MAC** → ton ordinateur
- **Target IP** → l'ordinateur dont tu veux connaître le MAC
- **Target MAC** → inconnu (000...00)

C'est comme crier dans la rue : "Qui habite au numéro 10 ?"

La réponse ARP (ARP Reply)

L'ordinateur avec l'IP recherchée répond : "Moi, **192.168.0.1** , mon MAC est **00:13:46:0b:22:ba** ".

- Cette réponse est envoyée **directement à ton ordinateur** (unicast).
- Dans cette trame, on voit :
 - **Sender IP/MAC** → l'ordinateur cible
 - **Target IP/MAC** → ton ordinateur

C'est comme si la personne au numéro 10 te montrait sa plaque : "Voilà ma plaque !"

Videz votre cache ARP, puis lancez une capture Wireshark avant de faire un ping vers une des machines de votre sous-réseau.

a) Vider le cache ARP

- **Windows :**

arp -d

- **Linux :**

sudo ip -s -s neigh flush all

b) Lancer Wireshark

- Capture sur l'interface réseau utilisée (Wi-Fi ou Ethernet).

c) Faire un ping

- Ping vers une machine de ton sous-réseau, par exemple :

ping 192.168.1.5

d) Créer un filtre pour retrouver la requête ARP

- Dans le filtre Wireshark :

arp.opcode == 1

- Explication :
 - arp.opcode == 1 → ARP request (demande)
 - arp.opcode == 2 → ARP reply (réponse)
- Tu peux aussi filtrer par IP :

arp.dst.proto_ipv4 == 192.168.1.5

- Cela montre uniquement les requêtes ARP destinées à l'IP que tu as pingée.

Créez un filtre pour retrouver la requête ARP qui vous a permis de récupérer l'adresse MAC de la machine visée.

Filtrer les requêtes ARP

- **ARP request** = requête qui demande : "Qui a cette IP ? Donnez-moi votre MAC."
- Dans le **filtre Wireshark**, tape :

arp.opcode == 1

- 1 = **request**, 2 = **reply**

Filtrer pour une IP cible précise

Si tu sais l'IP de la machine que tu as pingée, tu peux filtrer encore plus précisément :

`arp.opcode == 1 && arp.dst.proto_ipv4 == 192.168.1.5`

- `arp.dst.proto_ipv4` → l'IP que le PC veut résoudre
- Ici → seules les requêtes ARP **ciblant 192.168.1.5** apparaîtront.